Hochschule Furtwangen
Fakultät I
Prof. Dr. Lothar Piepmeyer

HOCHSCHULE FURTWANGEN UNIVERSITY HFU

Praktikum ‚Objektorientierte Programmierung'

Aufgabenblatt 4

Für dieses erste Aufgabenblatt müssen Sie zunächst eine C++ Entwicklungsumgebung wie etwa CLion aufsetzen und einrichten. Wie in der Vorlesung besprochen, empfehle ich die Compiler-Flags -Wall -Wextra. Um möglichst im Einklang mit der Vorlesung zu sein, sollten Sie dafür sorgen, dass Ihr Compiler den C++14-Standard unterstützt. In keiner der Aufgaben darf der *globale* Namensraum genutzt werden.

- Definieren Sie einen eigenen Namensraum.
- Die Anweisung `using namespace` darf nicht genutzt werden.
- Alle Namen, die in Ihrem Code auftreten sind einfache und klare englische Begriffe. Überhaupt sollte Ihr Code klar und einfach sein. Gehen Sie davon aus, dass es eine einfache und klare Lösung gibt. So sind lange if-else-Kaskaden sicher nicht nötig.
- Deklaration und Implementierung der Klassen müssen getrennt sein.

Aufgabe 1:
Die folgende Funktion wurde in der Vorlesung entwickelt:

```
int gcd(int v, int w){
    while(w!=0){
        int old_w=w;
        w=v%w;
        v=old_w;
    }
    return std::abs(v);
}
```

a. Der folgende Code gibt 0 aus:

```
std::cout << gcd(0,0) << std::endl;
```

Dieses Ergebnis ist falsch. Machen Sie sich Gedanken darüber, warum es für diesen Funktionsaufruf kein sinnvolles Ergebnis gibt.

b. Solche Probleme können sehr gut mit Exceptions gelöst werden:

```
int gcd(int v, int w){
  if (v==0 && w==0)
    throw std::invalid_argument("arguments must not be equal zero");
    while(w!=0){
        int old_w=w;
        w=v%w;
        v=old_w;
    }
  return std::abs(v);
}
```

Was passiert jetzt, wenn man die Anweisung aus a. ausführt?

c. Exceptions können gefangen werden. Probieren Sie es aus:

```
try {
  std::cout << gcd(0, 0) << std::endl;
} catch (...) {
  std::cout << "call to gcd failed..." << std::endl;
}
```

Nutzen Sie `assert` geschickt, um einen Test für `gcd` zu entwickeln, der auch den Fall prüft, dass eine Exception geworfen wird, wenn beide Parameter den Wert 0 haben. Das ist schwieriger, als man zunächst glaubt, weil die Exception hier *nicht* zum Abbruch des Programmes führen darf. Der Test soll im Gegenteil fehlschlagen, wenn keine Exception geworfen wird. Machen Sie sich die Logik dieser Vorgehensweise klar.

Aufgabe 2:
Entwickeln Sie unter Verwendung der Klasse `Position`, die Sie bereits entwickelt haben, eine Klasse `City`, die neben einem Konstruktor die folgende Schnittstelle implementiert:

```
// jede Stadt hat einen Namen
getName()

// jede Stadt hat eine x-Koordinate
getX()

// jede Stadt hat eine y-Koordinate
getY()

// jede Stadt hat Sehenswürdigkeiten (Points of Interest), auf die man
zugreifen kann
getPOI(int i)

// jede Stadt hat Sehenswürdigkeiten (Points of Interest), die man
ändern kann
setPOI(int i, ???)

// es gibt je Stadt eine bestimmte Zahl an Sehenswürdigkeiten.
getNumberOfPOIs()
```

Die Rückgabetypen wurden absichtlich nicht näher spezifiziert. Ein Parameter wurde absichtlich mit '???' gekennzeichnet. Alle weiteren Details ergeben sich aus den Tests, die Sie in der beiliegenden Datei `test_city.cpp` finden. Lesen Sie die Tests sorgfältig, bevor Sie an die Arbeit gehen. Ich habe meinen Namespace `hfu` genutzt, Sie können Ihren eigenen Namespace verwenden. Weitere Änderungen an den Test sind nicht zulässig. Arbeiten Sie redundanzarm.